Inherited Mathematical Foundations

Companion to *A Smaller Checked Presentation of iPRA*

Attributed source compilation, November 11, 2026

This supplement compiles the mathematical chain inherited by the smaller checkers. The first part is reproduced from the mathematical sections of the incoming guarded-source report. The appendices reproduce the earlier guarded and scalar foundations supplied with it. These are *not new mathematical results of the present package*. Historical phrases such as "new", "current", or "proposed" inside the reproduced sections refer to their original reports.

The source snapshot is ProveIt commit `a4e5e96c456303cf231eab2aeeb3291d8889833f`, directory `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`; the immediate source is the archive `ipra_guarded_source_2026-11-11.zip` in `docs/incoming/`. Its full original report source is preserved as `reference/inherited_report.tex`. Every reproduced foundation module is preserved in `reference/foundations/`. The repository's MIT No Attribution license is included. Citation aliases in this compilation identify the inherited reports and do not assert an independent literature priority for their contents.

The new checker changes and their finite tests are in the main report. This supplement is written mathematics. It is not a fresh build of the upstream Lean development or a formal verification of the new Python modules. The arity appendix describes its original predecessor protocol, including the old introduction-choice convention. Its theorem is used together with the guarded composition and the main report's preservation arguments; it is not presented as a literal description of every new checker line. In particular, the main report explains why computation-supply specialization is performed after translation into ordinary iPRA.

Contents

1 The complete proposed presentation **4**
1.1 Terms, public formulas, and auxiliary formulas 4
1.2 All primitive axioms and rules 4

2 Soundness and the purpose of the guard **5**

3 The equational dependency theorem **6**
3.1 Equation cases and safe equality 7
3.2 Two unconditional tests 7
3.3 Reversing an equality whose source is zero 8

4 Completeness of the combined presentation **8**

5 The two interpretations and both composites **9**
5.1 The arithmetic bridge 9
5.2 The logical bridge and guarded padding 10

A Complementary introductions with one local assumption **12**
A.1 Complete native syntax and rule inventory 12

A.2 Exact equivalence and the unused-witness construction . . . 13
A.3 Induction and numerical cases separate cleanly . . . 14
A.4 The entire ordinary logical interface . . . 15
A.5 Whole-domain realizations and both formula composites . . . 16
A.6 What the theorem does and does not claim . . . 17

B A guarded core with a deterministic introduction rule 17
B.1 Complete syntax, axioms, and inference rules . . . 17
B.2 Padding, equation implication, and numerical cases . . . 18
B.3 Recovery of contextual equation induction . . . 20
B.4 Exact translations to and from the inherited calculus . . . 20
B.5 Why an unrestricted checker can still be sound . . . 22
B.6 Why merely deleting the choice would not suffice . . . 22

C Internal scalar predecessor bootstrap 23
C.1 Elementary terms and two finite cycles . . . 23
C.2 A ramp with a global scaling identity . . . 24
C.3 A comparator from scalar iteration . . . 25
C.4 Saturated iteration gives predecessor . . . 25
C.5 Dependency boundary . . . 27

D From literal-equation induction to full intuitionistic PRA 27
D.1 The exact hypotheses . . . 27
D.2 Constructor separation and two elementary tests . . . 28
D.3 Four elementary uses of equation induction . . . 29
D.4 An equation invariant that reflects actual equality . . . 30
D.5 Compilation of all quantifier-free formulas . . . 31
D.6 Recovery of induction and the full theory . . . 32

E Pairing, primitive recursion, and internal numerical composites 32
E.1 Pairing from a bounded square-root search . . . 32
E.2 Compiling each primitive recursion . . . 33
E.3 Why both numerical round trips are internal . . . 34

F Arithmetic strength and both ordinary interpretation composites 34
F.1 The arithmetic dependency order . . . 35

G A uniform arity encoding and its checking theorem 36
G.1 The complete sorted interpretation of arrays . . . 36
G.2 One depth calculation replaces the lookup table . . . 37
G.3 Opening and scope . . . 37

G.4 A computation hypothesis only needs a zero instance 38
G.5 Audit of the logical continuations 39
G.6 Canonical completeness, including zero witnesses 40
G.7 JSON and finite-runtime qualifications 41

1 The complete proposed presentation

1.1 Terms, public formulas, and auxiliary formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in h; it is not bound in n or a. The body h may contain fixed numerical parameters. Terms do not contain a numerical variable ranging over program descriptions, and no uniform evaluator is postulated. Write $1 = S0$, $2 = S1$, and

$$C(v, a, b) := I(v, a; u.b),$$

where the update variable is unused. All binders use capture-free substitution and may be renamed.

Definition 1.1 (Public language)**.** A canonical public formula is a literal equation $s = t$, or a formula $D_{x,y}B$ satisfying one of the following two clauses:

1. B is a compound canonical formula, so its root is D;
2. B is exactly of the guarded atomic shape

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{G}$$

The bodies h, k may depend on x, y, u and passive parameters. In particular, the guard is a syntactic condition, not a test for the theoremhood of a base case. Both sides in (G) compute to zero at $y = 0$. The designated contradiction is the public equation $0 = 1$.

The finite auxiliary formula language used *inside proofs* is larger:

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A. \tag{A}$$

It permits unguarded atomic matrices. A retained context is a finite reusable list of auxiliary formulas. A public sequent has a canonical context and canonical conclusion, but its proof may use (A) internally. This convention is part of the construction, not an omitted restriction. There is no claim below about forcing every intermediate formula to satisfy the public guard.

1.2 All primitive axioms and rules

In every context the two nonlogical axiom schemas are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

There are exactly the following four inference patterns besides those two computation schemas.

Hypothesis.

$$\frac{}{\Gamma \vdash A} \qquad (A \in \Gamma). \tag{Hyp}$$

Elimination selected by the source. For any fixed auxiliary formula template $F(v)$, put

$$M(s,t;F) := \begin{cases} Q_v F(v), & s \equiv 0, \\ s = t, & s \not\equiv 0. \end{cases}$$

The rule is

$$\frac{\Gamma \vdash M(s,t;F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{SE}$$

Here $s \not\equiv 0$ means that the source is not literally the one-node term 0. A variable or an iterator that evaluates to zero is still a permitted equality source. The $s \equiv 0$ branch is ordinary anchored Q elimination; the other branch is restricted equality transport.

Paired elimination.

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenparameter b is fresh for Γ, A, and the passive parameters of B.

One deterministic paired introduction. Define $\eta_B(z) = z$ when B is a literal equation and $\eta_B(z) = 0$ otherwise. Then

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}. \tag{DI}$$

The eigenparameter z is fresh for Γ, t, and the passive parameters of B. The local assumption is discharged. There is no certificate choice between cases and progress: the root of the fixed matrix determines it. Atomic matrices use progress; compound matrices use their zero instance.

Call this auxiliary proof calculus $\mathcal{U}^-$, and call its consequence relation on public canonical sequents the proposed presentation. There are no additional primitive axioms or rules for reflexivity, symmetry, successor injectivity, successor separation, numerical coverage, induction, propositional connectives, or ordinary quantifiers. Weakening, assumption cut, and fresh renaming are admissible by induction on derivations. Arbitrary numerical substitution into a restricted proof is not assumed in the bootstrap argument.

2 Soundness and the purpose of the guard

Interpret the terms in iPRA by their primitive recursive functions, one fixed finite iterator description at a time. Equality is unchanged. Give auxiliary formulas the following compositional reading, denoted by a superscript $*$:

$$\begin{aligned} (s = t)^* &= (s = t), \\ (Q_y B)^* &= B^*(0) \to \forall y\, B^*(y), \\ (D_{x,y}B)^* &= \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ a literal equation}, \\ \exists x\, (B^*(x,0) \to \forall y\, B^*(x,y)), & B \text{ compound}. \end{cases} \end{aligned}$$

This is a specified syntax-sensitive translation. It is not an assertion that a single ordinary connective has interchangeable meanings when its matrix is replaced by a logically equivalent formula.

Lemma 2.1 (Auxiliary soundness)**.** *If $\Gamma \vdash_{\mathcal{U}^-} A$, iPRA proves the corresponding consequence from Γ^* to A^*.*

Proof. Computation and hypotheses are immediate. Numerical substitution preserves the root constructor of a formula, so the reading commutes with numerical substitution; the equality branch of SE is sound. Its zero branch is anchored Q elimination.

At an atomic DI, the premise proves each progressive implication, and universal introduction followed by existential introduction proves the displayed atomic reading. At compound DI, numerical coverage supplies the missing zero-or-successor split, yielding the anchored reading.

At an atomic DE, a progressive equation predicate gives $E(0) \to \forall y\, E(y)$ by contextual equation induction in iPRA. This is exactly the exported Q resource. Compound DE is ordinary existential elimination. All eigenvariable conditions are the standard ones. Induction on the finite derivation completes the proof. □

The public ordinary reading, denoted by A°, instead reads every public D uniformly as

$$(D_{x,y}B)^\circ = \exists x\, (B^\circ(x,0) \to \forall y\, B^\circ(x,y)). \tag{O}$$

Lemma 2.2 (Agreement on the public grammar)**.** *For every canonical formula A, iPRA proves $A^* \leftrightarrow A^\circ$.*

Proof. At a guarded atomic matrix $E(x,y)$, computation proves $E(x,0)$. Consequently, with x fixed,

$$\forall y\, (E(x,y) \to E(x,Sy)) \quad \longleftrightarrow \quad E(x,0) \to \forall y\, E(x,y).$$

The forward implication is equation induction; the reverse uses the theorem $E(x,0)$ to obtain all instances. Existential quantification preserves the equivalence. At a compound matrix the two readings have the same form, so structural induction applies. □

The guard is essential to this argument. Without a true zero instance, a progressive predicate and an anchored predicate need not agree. For example, the equation asserting that the parity iterator is 1 is false at zero, true at one, and false at two. Its anchored closure is true, while its progressive closure is false. We make no completeness or composite claim for all unguarded auxiliary formulas.

3 The equational dependency theorem

This section isolates a smaller set of hypotheses for the repository's literal-zero-source argument [2]. The singleton test and the zero-filter construction are inherited from that argument. The dependency statement, and its use in the combined presentation, are the point here.

Definition 3.1 (Equational fragment)**.** Let $\mathcal{E}^-$ have only equation formulas, hypotheses, the two computation schemas, equation-template transport

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad (s \not\equiv 0,\ F \text{ a literal equation template}), \tag{Tr$^-$}$$

and contextual equation induction

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(r)}. \tag{EI}$$

The eigenparameter z is absent from $\mathrm{FV}(\Gamma)$, the target r, and all passive parameters of E; these remain fixed during induction.

Theorem 3.2 (Equational source elimination)**.** *Unrestricted equation-template transport is admissible in $\mathcal{E}^-$. Thus the restriction $s \not\equiv 0$ changes neither its closed equational theorems nor its contextual equational consequences. No logical binder or logical case introduction is needed.*

3.1 Equation cases and safe equality

Equation cases already follow from EI: given $E(0)$ and $E(Sz)$ in a fixed context, weaken the successor proof by the unused assumption $E(z)$ and apply EI at the desired target. This observation removes the logical case-introduction dependency from the original proof.

Reflexivity of an arbitrary r follows by taking $b = I(0, r; u.0)$ and using $b = r$ as both transport premises with motive $v = r$. The source b is allowed. From $p = q$ with $p \not\equiv 0$, permitted transport gives both $q = p$ and $H(p) = H(q)$ for every numerical template H. All computation equations have an iterator on the left and can therefore be reversed as equations. Their reversed equations can be used as transport majors only when the new left side is syntactically nonzero.

Lemma 3.3 (Joining through a represented zero)**.** *If $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$, then Γ proves $p = q$ using computation and restricted transport alone.*

Proof. Put $z_0 = I(0, 0; u.0)$ and $b_p = I(0, p; u.0)$. Transport $b_p = b_p$ along $p = 0$ in the left-hand occurrence of its initializer, obtaining $z_0 = b_p$. Reverse at the permitted source z_0, then use $b_p = p$ to get $p = z_0$. Similarly derive $q = z_0$. Reverse the latter at source q and transport through z_0 to obtain $p = q$. Every source used is one of p, q, z_0, b_p, b_q, hence is permitted. This avoids transitivity through literal zero before that operation has been established. □

3.2 Two unconditional tests

Define the following finite scalar terms:

$$\begin{aligned} Z(v) &= C(v, 1, 0), & K(v) &= C(v, 0, 1), & H(v) &= C(v, 1, 2), \\ N(n) &= I(n, 0; u.Z(u)), & R(n) &= I(n, 0; u.H(u)), & L(n) &= N(R(n)). \end{aligned}$$

These are explicit abbreviations over $0, S, I$, not new function symbols.

Lemma 3.4 (Test and filter identities)**.** *The equational fragment proves, at every term target v,*

$$L(1) = 1, \qquad L(SSv) = 0, \qquad L(Sv) = Z(v), \tag{T}$$

and, at every term target t,

$$t = C(Z(t), t, 0). \tag{F}$$

Proof. Equation cases prove $H(v) = SK(v)$: the zero endpoints compute to 1 and the successor endpoints compute to 2. These joins use the nonzero numerals $1, 2$. Computation gives $N(1) = 1$, $N(2) = 0$, and $R(1) = 1$. Also

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Replacing the iterator source $R(Sv)$ in $R(SSv) = H(R(Sv))$ now gives $R(SSv) = 2$. Hence $L(1) = 1$ and $L(SSv) = 0$ by replacement at iterator sources inside N.

For $L(Sv) = Z(v)$, use equation cases. At zero both endpoints equal 1. At a successor both equal zero, and both left-hand expressions are iterator terms, so the preceding joining lemma applies. The case rule itself supplies every arbitrary target; no substitution into a proof with a variable transport source is required.

For the filter, apply equation cases to $v = C(Z(v), v, 0)$. At zero the right side computes to $C(1, 0, 0) = 0$; at Sv it computes to $C(0, Sv, 0) = Sv$. Reverse these equations at their iterator sources to obtain the required branch orientations. Both branch contexts are empty. □

3.3 Reversing an equality whose source is zero

Lemma 3.5 (Zero-source symmetry)**.** *For every term t, $0 = t \vdash_{\mathcal{E}^-} t = 0$.*

Proof. If $t \equiv 0$ syntactically, use reflexivity. Otherwise fix t and retain the assumption $0 = t$. Put $A(n) = I(n, t; u.Su)$ and induct on

$$E(n) \equiv n = A(n).$$

At the base, computation gives $A(0) = t$, and safe symmetry gives $t = A(0)$. Transport in the retained $0 = t$ at the permitted source t gives $0 = A(0)$.

At the step, $z = A(z)$ has a variable source, which is permitted. Successor congruence gives $Sz = SA(z)$. Reverse the computation $A(Sz) = SA(z)$ and transport at source $SA(z)$ to obtain $Sz = A(Sz)$. The parameter t and the context remain fixed.

Apply EI directly at 1. It gives $1 = A(1)$; forward computation of $A(1)$ gives $1 = St$. Congruence for L, followed by (T), gives $1 = Z(t)$; safe symmetry at source 1 gives $Z(t) = 1$. Weaken the unconditional filter (F) into the retained context. Replace $Z(t)$ by 1 there, then compute $C(1, t, 0) = 0$. These last sources are iterators, and the result is $t = 0$. □

Completion of Theorem 3.2. Full symmetry now follows by the safe-source case and the preceding lemma. For congruence from $0 = t$ with $t \not\equiv 0$, first reverse it, use permitted congruence at t, and reverse the resulting equality. The literal-zero/literal-zero case uses reflexivity.

For transitivity $r = s$, $s = t$, a nonzero middle source is already allowed. If s is literal zero and one endpoint is literal zero, a premise gives the conclusion. Otherwise reverse $0 = t$ and join $r = 0$, $t = 0$ by the represented-zero lemma. Thus all numerical equality algebra is available. For $F(v) \equiv A(v) = B(v)$, use numerical congruence on both sides and join with $A(s) = B(s)$ to obtain $A(t) = B(t)$. □

The proof uses weakening, cut, renaming, and only source-preserving helper substitutions. The one fresh variable used as a transport source in the induction step is never replaced by literal zero inside that derivation. Arbitrary targets are supplied by the induction or case rule itself. This distinction is necessary for a noncircular proof.

4 Completeness of the combined presentation

Lemma 4.1 (Equation induction inside $\mathcal{U}^-$)**.** *Contextual EI is derivable in $\mathcal{U}^-$ without using transport.*

Proof. Given a progressive step for $E(y)$, take an unused witness coordinate x. Atomic DI with witness 0 gives $D_{x,y}E(y)$. DE opens $Q_yE(y)$ independently of its fresh witness, and the zero-source branch of SE applies it to the supplied base $E(0)$ at the desired target. □

This lemma may use an *unguarded auxiliary* matrix E. It is why the proof language was stated explicitly. Applying Theorem 3.2, with additional retained auxiliary assumptions treated as passive context, restores all equation-template transport in $\mathcal{U}^-$ before any logical padding is needed.

Let $\mathcal{G}$ denote the inherited guarded-core calculus: the same public guarded formulas, the same deterministic introduction, and unrestricted transport on canonical templates, with anchored Q resources opened by DE. The repository supplies effective translations, both formula composites, and a whole-domain bi-interpretation of $\mathcal{G}$ with iPRA [3].

Lemma 4.2 (Transport on every canonical template)**.** *For a canonical template $F(v)$, the unrestricted rule $s = t, F(s) \vdash_{\mathcal{U}^-} F(t)$ is admissible.*

Proof. Induct externally on the formula template, uniformly in retained contexts. The equation case was just established. Suppose $F(v) = D_{x,y}B(v, x, y)$. Open $F(s)$ at a fresh witness a.

If B is guarded atomic, computation and the restored equality algebra prove $B(s, a, 0)$. Its opened Q resource therefore yields $B(s, a, Sz)$. Equation transport gives $B(t, a, Sz)$. Apply the deterministic atomic introduction to the target at witness a, ignoring its local assumption $B(t, a, z)$, and close DE.

If B is compound, introduce the target by the deterministic case rule. Use the smaller-template induction hypothesis and $t = s$ to turn its local base $B(t, a, 0)$ into $B(s, a, 0)$. Apply the source Q resource at Sz, and use the induction hypothesis forward to obtain $B(t, a, Sz)$. Close case introduction and DE. All newly chosen parameters are fresh for the retained equality and context. Passive substitutions preserve the guard. □

In the inherited clause presentation, the canonical formulas in Γ are read as ordinary assumptions, represented by their constant clauses.

Theorem 4.3 (Exact public consequence relation)**.** *For every finite context Γ of canonical formulas and every canonical A,*

$$\Gamma \vdash_{\mathcal{U}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{G}} A.$$

Proof. A guarded-core proof translates to $\mathcal{U}^-$: its computations, hypotheses, introductions, and resource eliminations are available, while each unrestricted canonical transport is replaced by Lemma 4.2. Starting from public formula contexts, the reachable clauses are constant clauses for ordinary assumptions and anchored clauses opened by DE. The former correspond to hypotheses, and each use of the latter corresponds to a Q elimination. No arbitrary additional clause context is being identified with a Q context.

Conversely, auxiliary soundness translates a $\mathcal{U}^-$ proof to iPRA. Agreement of the two readings on the canonical context and conclusion gives its ordinary guarded reading. The inherited faithful guarded/iPRA translation, including its canonical formula composite, returns a guarded proof of the original sequent. This is an effective translation of finite proofs using the inherited translations. □

The converse does not identify all auxiliary formulas with their ordinary anchored readings. Only the public context and conclusion use the agreement lemma. Likewise, Lemma 4.2 does not assert full transport or full formula congruence for every unguarded auxiliary binder.

5 The two interpretations and both composites

The consequence theorem transfers an existing bi-interpretation, rather than merely transferring consistency. Here are its components and the exact facts needed from the inherited foundations.

5.1 The arithmetic bridge

The ordinary scalar theory T_{eq} is intuitionistic first-order logic with equality over the iterator terms, the two computation equations, coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, separation $0 = 1 \to \bot$, and contextual induction for literal equations. Its inherited construction derives predecessor, equality reflection into a Boolean term, quantifier-free induction, pairing with projections, and compilation of every fixed primitive recursive definition, in that order [4, 5].

The direction to iPRA interprets each fixed iterator body by primitive recursion. In the reverse direction, a description of a primitive recursive function is compiled to an iterator term; a counter and

the current result can be packed into the scalar state. The translations prove both term round trips equal to the original terms. They are interpretations over the whole numerical domain with actual equality, not a restriction to standard numerals or a quotient by an external equivalence relation.

The pinned Lean modules `BiInterpretation`, `NativeBiInterpretation`, and their dependencies state the two directions and the round trips for the ordinary scalar and native theories. The guarded grammar and padding bridge remain written mathematics in the pinned research package. This report does not claim that the present composition has been checked by Lean.

5.2 The logical bridge and guarded padding

The inherited native core has equations, Q, and D. Write $\mathrm{sel}(v, A, B)$ for its formula selector, equivalent to A at $v = 0$ and to B at successors, and $\mathrm{gate}(v, A)$ for its formula gate, true at zero and equivalent to A at successors. Their equational leaves use the scalar selector C; syntactic padding aligns compound shapes. The ordinary logical operations are macros, for example

$$\begin{aligned} \bot &:= 0 = 1, \\ A \to B &:= Q_y\, \mathrm{sel}(y, A, B), \\ A \wedge B &:= Q_y\, \mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\ \exists x\, A(x) &:= D_{x,y}\, \mathrm{gate}(y, A(x)), \\ A \vee B &:= \exists v\ \mathrm{sel}(v, A, B), \\ \forall x\, A(x) &:= A(0) \wedge Q_y A(y), \end{aligned}$$

where the parity iterator is a fixed scalar term. All natural deduction rules for these macros are inherited derived rules; they are not additions to Section 1.

To pass explicitly to the equivalent one-object-binder presentation, recursively replace each $Q_y B$ by $D_{u,y} B$ with a fresh unused u. The two are interderivable in the old core: paired elimination gives Q from the dummy-witness D, while case introduction and QE give the reverse direction with witness 0. After this expansion, Q is needed only as a context resource. For an equation E, put

$$G_v E = (C(v, 0, s) = C(v, 0, t)) \quad (E \equiv s = t), \qquad L(E) = D_{u,v} G_v E,$$

with fresh u, v, and let $P(E) = L(E)$, $P(A) = A$ at compound formulas. Guarded padding gives $P(A) \dashv\vdash A$. The translation from the unrestricted one-binder core to the guarded grammar is

$$T(s = t) = s = t, \qquad T(D_{x,y} B) = D_{x,y} P(TB).$$

It commutes with numerical substitution up to renaming. The inherited guarded-core argument proves both $JT(A) \dashv\vdash A$ in the old core and $TJ(B) \dashv\vdash B$ for canonical guarded B, where J is inclusion. The guarded atomic case of the second composite uses its true base; it is not an unrestricted congruence assertion for all auxiliary matrices.

Theorem 5.1 (Bi-interpretability and equiconsistency)**.** *The proposed public presentation, realized by* (O)*, is bi-interpretable with full first-order iPRA. Both numerical domains are whole and both numerical composite isomorphisms are given by $x = y$. Both formula composites are provably equivalent to the original formulas. In particular,*

$$\varnothing \vdash_{\mathcal{U}^-} 0 = 1 \quad \Longleftrightarrow \quad \mathrm{iPRA} \vdash \bot.$$

Proof. Let Φ be the inherited iPRA-to-guarded translation, obtained by composing the scalar compiler, the native logical translation, the dummy-witness expansion of Q, and guarded padding. Let Ψ be the

guarded ordinary reading followed by the scalar-to-iPRA interpretation. Use these same maps for the new presentation. Theorem 4.3 preserves exactly every canonical consequence used by Φ, Ψ, and their guarded composite proofs.

The inherited iPRA composite proves $\varphi \leftrightarrow \Psi\Phi(\varphi)$ for every ordinary formula φ. The inherited guarded composite proves $A \dashv\vdash \Phi\Psi(A)$ for every canonical A; the consequence theorem makes this a proof in the proposed presentation. On numerical domains nothing has changed, and inherited term round-trip equations witness the literal identity graph. Finally the designated contradiction has the same reading throughout, giving the displayed equivalence. □

This proof uses the inherited full bridge as a proved input, with its sources supplied for inspection. A new derivation of predecessor or a new primitive-recursion compiler is neither needed nor claimed.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y) \rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0,t;u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t, u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the

two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\,(x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their

selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B A guarded core with a deterministic introduction rule

The change is to the public formula language and the introduction rule, not to the numerical domain. An atomic matrix immediately below a paired binder must have a fixed, syntactically recognizable zero guard. Such a matrix has a provable zero instance. Consequently it needs only the progress introduction; the separate atomic case introduction can be deleted. Compound matrices retain the case introduction.

B.1 Complete syntax, axioms, and inference rules

Numerical terms are those of the inherited scalar calculus:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the displayed variable u is bound in h. There is one numerical sort, fixed finite term syntax, and capture-free substitution. Put

$$C(v, a, b) := I(v, a; u.b), \qquad 1 := S0,$$

where u is fresh. For a literal equation $E \equiv (s = t)$, define

$$G_v E \;:=\; \big(C(v, 0, s) = C(v, 0, t)\big). \tag{Guard}$$

The displayed abbreviations expand into terms of the given syntax.

Call a literal equation a *zero-initialized matrix in y* when it has the syntactic form

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{Canonical atomic matrix}$$

Here h, k may depend arbitrarily on the state variable u, on x, y, and on fixed parameters; state binders are renamed as necessary. The selector guard $G_y E$ is a special case, with state-independent updates. Every matrix of (Canonical atomic matrix) has the theorem $B(x, 0)$ by the two zero computations and equality.

The canonical formulas of $\mathcal{G}$ are generated as follows:

1. Every literal equation $s = t$ is a formula.
2. $\mathsf{D}_{x,y}B$ is a formula when B is already a compound canonical formula (its root is D).
3. $\mathsf{D}_{x,y}B(x, y)$ is a formula whenever B has the zero-initialized shape (Canonical atomic matrix).

The bound variables x, y are distinct. Rule 3 is the restriction on atomic matrices. It does not require an equality decision procedure or a proof that an arbitrary base is true: the guard has the displayed literal shape. The designated contradiction is $0 = 1$.

Retained contexts contain reusable clauses $\langle A; y.B\rangle$, where A, B are canonical formulas and $y \notin \mathrm{FV}(A)$. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where v is fresh and H is the fixed computation theorem $I(0, 0; u.0) = 0$.

The two nonlogical axiom schemas, available in every context, are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0,IS}$$

Besides these, there are precisely the following rules:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } \langle A; y.B(y)\rangle \in \Gamma \qquad \text{(Use)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

and the single, root-determined introduction schema

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{DI$_{\mathrm{root}}$}$$

There is no certificate choice between these two cases. At an atomic matrix the guard condition on the conclusion is mandatory in the canonical calculus. In DE, a is fresh for Γ, F and the fixed parameters. In DI, z is fresh for Γ, t and the fixed parameters. The displayed local resources are discharged. Transport ranges over canonical formula templates, and capture-free numerical substitution preserves their guard shape.

There are no additional primitive logical rules, reflexivity axioms, successor axioms, coverage axioms, or induction schemas. Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on derivations. For cut, replace a use of the corresponding constant clause by the supplied proof, renaming eigenparameters first.

B.2 Padding, equation implication, and numerical cases

Write $A \dashv\vdash B$ for interderivability under arbitrary additional retained context. The following preliminary arguments use no contextual induction rule.

Lemma B.1 (Equality and guard endpoints)**.** *Reflexivity, symmetry, transitivity, numerical congruence, and their usual contextual uses are derivable. Moreover,*

$$\vdash G_0 E, \qquad G_{Sz} E \dashv\vdash E.$$

Proof. For reflexivity, put $r = I(0, t; u.0)$. The computation equation $r = t$ serves as both premises of transport with motive $v = t$, yielding $t = t$. Standard equality motives then give the remaining equality rules. The two guard claims follow from the two computation equations for C, together with equality transport. □

For an equation E let

$$L(E) := \mathsf{D}_{u,v} G_v E, \qquad P(A) := \begin{cases} L(A), & A \text{ is an equation,} \\ A, & A \text{ is compound.} \end{cases} \tag{Padding}$$

The newly bound u, v are fresh for E. In particular, $P(A)$ is always compound.

Lemma B.2 (Padding)**.** *$L(E) \dashv\vdash E$, and therefore $P(A) \dashv\vdash A$.*

Proof. Under E, atomic progress introduction derives $L(E)$: its target $G_{Sz}E$ follows from the retained E, independently of the local induction assumption. Conversely, DE on $L(E)$ opens $\langle G_0 E; v.G_v E\rangle$. Supply the guard-zero theorem and use the clause at 1; guard removal gives E. □

We will need to reason by numerical cases under an assumption that contains the case variable. It is essential not to violate the eigenvariable condition by directly retaining that assumption during case introduction. A small implication macro resolves this issue.

For equations $U \equiv (s = t)$ and $V \equiv (r = q)$ put

$$M_w(U, V) := (C(w, s, r) = C(w, t, q)), \qquad U \Rightarrow_g V := \mathsf{D}_{a,w} L(M_w(U, V)), \tag{Equation implication}$$

with fresh padding variables and unused a. The matrix of the outer binder is compound.

Lemma B.3 (Implication between equations)**.** *The macros in* (Equation implication) *admit both rules*

$$\frac{\Gamma, U \vdash V}{\Gamma \vdash U \Rightarrow_g V}, \qquad \frac{\Gamma \vdash U \Rightarrow_g V \qquad \Gamma \vdash U}{\Gamma \vdash V}.$$

Proof. The endpoint identities for C and padding give $L(M_0(U, V)) \dashv\vdash U$ and $L(M_{Sz}(U, V)) \dashv\vdash V$. Compound case DI therefore gives introduction. DE, the first endpoint, Use at 1, and the second endpoint give elimination. These endpoint conversions use numerical transport with parameters outside the fresh padding binders; they do not assume a binder-congruence theorem. □

Lemma B.4 (Numerical cases)**.** *For every canonical $A(y)$ the following rule is admissible:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{NC}$$

Proof. First suppose A is compound. Case DI with an unused witness coordinate introduces $\mathsf{D}_{u,y} A(y)$, ignoring its local base assumption in the successor premise. DE opens $\langle A(0); y.A(y)\rangle$; supply the given base proof and use it at t.

If A is an equation, apply this already proved compound case rule to $L(A(y))$. Padding converts the two premises to its endpoints and converts the conclusion back. No progress induction is being used in this construction beyond the special padding introductions. □

B.3 Recovery of contextual equation induction

Theorem B.5 (Equation induction)**.** *For every literal equation $E(y)$, $\mathcal{G}$ derives*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{EI}$$

Proof. Put $J(y) = G_y E(y)$. We first prove the progress premise $\Gamma, J(z) \vdash J(Sz)$ without making an invalid contextual use of NC. Consider the compound formula

$$\Phi(y) := J(y) \Rightarrow_g E(Sy).$$

Under Γ, prove $\Phi(0)$ by equation-implication introduction: ignore its local hypothesis, use the supplied proof of $E(0)$, and apply the supplied step proof specialized at 0 to obtain $E(1)$.

For fresh w, prove $\Phi(Sw)$ by equation-implication introduction. Its local hypothesis $J(Sw)$ gives $E(Sw)$ by guard removal, and the supplied step proof specialized at Sw gives $E(SSw)$. These two branch proofs have retained context Γ, independent of the case variable. Thus NC legitimately gives $\Phi(z)$ under Γ. Under the local hypothesis $J(z)$, equation-implication elimination gives $E(Sz)$, hence $J(Sz)$ by the successor guard endpoint.

Atomic progress DI now introduces $\mathsf{D}_{u,y}J(y)$ with unused u. DE and the theorem $J(0)$ yield $J(Sw)$, and hence $E(Sw)$, for a fresh w. Finally NC for E uses the supplied base $E(0)$ and these successor instances to conclude $E(t)$.

The only induction-like premise used here is the single displayed guarded progress introduction. In particular, no predecessor, coverage disjunction, formula induction, or pre-existing EI is assumed. □

B.4 Exact translations to and from the inherited calculus

Let $\mathcal{C}$ be the inherited Choice-DI calculus: the same scalar terms, arbitrary formulas $s = t$ and $\mathsf{D}_{x,y}B$, the same computation, Use, DE, and transport rules, with case DI at every matrix and progress DI at literal equations. Define $T : \mathcal{C} \to \mathcal{G}$ on finite formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}P(TB). \tag{Translation}$$

Each padding binder uses fresh names. This is a computable structural translation, preserves free numerical variables, commutes with capture-free numerical substitution up to bound-variable renaming, and translates every paired matrix to a compound matrix. Translate clauses by $T\langle A; y.B\rangle = \langle TA; y.TB\rangle$. Let $J : \mathcal{G} \to \mathcal{C}$ be literal inclusion.

Theorem B.6 (Translation of derivations)**.**

$$\Gamma \vdash_{\mathcal{C}} A \Longrightarrow T\Gamma \vdash_{\mathcal{G}} TA, \qquad \Gamma \vdash_{\mathcal{G}} A \Longrightarrow J\Gamma \vdash_{\mathcal{C}} JA.$$

The transformations are effective on finite derivations.

Proof. Inclusion is immediate: every guarded rule is an allowed Choice-DI rule. For T, induct on a Choice-DI derivation.

Computation is unchanged. Use commutes with substitution, and transport uses the translated motive. A case DI premise translates to a proof from $TB(t, 0)$ to $TB(t, Sz)$. Under the local assumption $P(TB)(t, 0)$, unpad, apply that translated proof, and pad the result. Compound case DI then proves the required $\mathsf{D}_{x,y}P(TB)$.

For a progress DI with equation matrix E, its translated step is still $\Gamma, E(t,z) \vdash E(t,Sz)$. To introduce $\mathsf{D}_{x,y}L(E(x,y))$, use compound case DI. Its retained base $L(E(t,0))$ supplies $E(t,0)$. The recovered contextual EI, under that base, gives $E(t,Sz)$, which padding converts to the target $L(E(t,Sz))$.

For DE, the translated major is $\mathsf{D}_{x,y}P(TB)$. It opens the clause $\langle P(TB)(a,0); y.P(TB)(a,y)\rangle$. The recursively translated minor expects $\langle TB(a,0); y.TB(a,y)\rangle$. Replace every use of this expected clause by: pad its antecedent, use the opened clause at the same term, and unpad its consequent. This effective local proof replacement preserves the fresh witness condition and the conclusion. □

Lemma B.7 (Compound-matrix congruence)**.** *If canonical compound matrices B, C are uniformly interderivable, then $\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}C$ in $\mathcal{G}$.*

Proof. Open the source binder at a fresh witness a. Introduce the target by compound case DI at that witness. Convert its retained base to the source base, use the opened source clause at Sz, and convert the successor instance to the target instance. Close DE. The reverse conversion proves the converse. □

Lemma B.8 (The guarded atomic composite)**.** *For an admissible zero-initialized atomic matrix $B(x,y)$,*

$$\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}L(B) \quad \textit{in } \mathcal{G}. \tag{Atomic composite}$$

Proof. Forwards, open the source clause at a. Introduce the target by compound case DI: unpad its base, use the source clause at Sz, and pad the result.

Backwards, open the clause of $\mathsf{D}_{x,y}L(B)$ at a. The two zero computations prove $B(a,0)$, and padding proves $L(B(a,0))$. Thus the opened clause gives $L(B(a,Sz))$, and unpadding gives $B(a,Sz)$, independently of the local assumption $B(a,z)$. Atomic progress DI therefore introduces $\mathsf{D}_{x,y}B$ at witness a. Close DE. This argument uses the true base explicitly; it would fail for an arbitrary unguarded atomic matrix. □

Theorem B.9 (Both composites and bi-interpretability)**.** *For all $\mathcal{C}$-formulas A and all canonical $\mathcal{G}$-formulas B,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{G}} TJ(B).$$

Consequently $\mathcal{C}$ and $\mathcal{G}$ admit effective structural intertranslations preserving their full consequence relations, with both native formula composites and literal identity numerical translations.

Proof. In $\mathcal{C}$, padding is interderivable with its argument, and case introduction proves binder congruence for arbitrary matrices. Structural induction gives the first displayed equivalence.

For the second, induct on a canonical B. Equations are unchanged. At a compound matrix, the induction hypothesis and compound-matrix congruence apply. At a guarded atomic matrix, the statement is exactly (Atomic composite). The equivalences are uniform under retained assumptions, so they extend to clauses and sequent contexts by the same local replacement of Use instances.

Both interpretations leave the numerical sort, every term, and numerical equality untouched. Both numerical composites therefore have the identity graph $x = y$; no quotient, retract, numeral restriction, or extra coding sort is involved. □

For the first-order bi-interpretability statement, use the ordinary realization that reads every canonical $\mathsf{D}_{x,y}B$ uniformly as $\exists x(B(x,0) \to \forall y B(x,y))$ and leaves numerical equality and all scalar terms unchanged. Composing the displayed translations and composites with the inherited full-domain

bridge between $\mathcal{C}$ and iPRA yields a bi-interpretation of this ordinary realization of $\mathcal{G}$ with full first-order iPRA. The contradiction $0 = 1$ is unchanged, so equiconsistency follows as well. The inherited arithmetic bridge supplies the primitive-recursive compiler and the ordinary logical interpretation; this section does not claim those inherited constructions as new.

B.5 Why an unrestricted checker can still be sound

There is a useful, but strictly separate, administrative calculus $\mathcal{U}$. It permits arbitrary atomic paired matrices and uses the same root-determined DI, with ordinary Q-style clauses on DE. It is not claimed that all of $\mathcal{U}$ is bi-interpretable with iPRA.

Give arbitrary administrative formulas their following ordinary interpretations:

$$(s = t)^\dagger = (s = t),$$

$$(\mathsf{D}_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is a literal equation,} \\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound.} \end{cases} \quad \text{(Mixed administrative reading)}$$

Retained clauses keep the ordinary reading $A^\dagger \to \forall y\, B^\dagger$. If the raw codec also permits administrative Q-closures as formula-shaped objects, including under transport or a paired binder, extend this interpretation by $(Q_y B)^\dagger = B^\dagger(0) \to \forall y B^\dagger(y)$. Such a matrix is compound for root-determined DI. This adds no canonical object-formula constructor and preserves the following soundness argument.

Every $\mathcal{U}$ inference is sound in iPRA under this reading. Atomic DI directly proves its progressive reading; compound case DI uses numerical coverage. In atomic DE, contextual equation induction turns $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, exactly the clause that DE supplies. All other rules are immediate, including transport: numerical substitution does not change whether a matrix is an equation.

On canonical zero-initialized matrices, the mixed and ordinary readings agree provably in iPRA. Indeed $B(x,0)$ is a theorem, so, with x fixed,

$$\forall y\, (B(x,y) \to B(x,Sy)) \quad \longleftrightarrow \quad B(x,0) \to \forall y\, B(x,y).$$

The forward direction is equation induction; the reverse uses the true base to obtain every instance. Structural induction extends this agreement to all canonical guarded formulas.

Thus administrative proofs with unguarded intermediate formulas are conservative on canonical guarded conclusions: translate an accepted derivation into iPRA, use the canonical agreement, and translate back through the established bi-interpretation. Conversely, every canonical guarded derivation is already an administrative derivation.

This justifies a checker contract consisting of soundness and completeness for canonical guarded statements while allowing broader administrative certificates. It does *not* make the raw checker a validator of the guarded public grammar. It also does not assert completeness for every noncanonical encoding under the mixed reading. If a comparison requires rejection of all noncanonical public goals, the corresponding syntax validator must be implemented and counted.

B.6 Why merely deleting the choice would not suffice

Without the guarded canonical language, root-deterministic DI does not preserve the original native consequence relation. Let

$$p(y) = I(y, 0; u.C(u, 1, 0)), \qquad E(y) \equiv (p(y) = 1),$$

and leave the existential witness coordinate unused. Computation gives $p(0) = 0, p(1) = 1, p(2) = 0$. The original case introduction proves $\mathsf{D}_{x,y}E(y)$: its base gives $0 = 1$, from which equality transport and selector computation prove every equation.

By contrast, the progressive reading of this formula is false in the standard natural numbers, because $E(1)$ holds and $E(2)$ does not. The mixed interpretation proves that it is not a theorem of the unrestricted root-deterministic calculus. The guard, the explicit translations, and both composite proofs are therefore substantive parts of the reduction, rather than optional notation.

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (39) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,\big(a=b\vee(a=b\to\bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{42}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{43}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{44}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A=0\vee b_A=1)\quad\text{and}\quad(b_A=0\longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

F.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem F.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

G A uniform arity encoding and its checking theorem

The arithmetic and object logic remain the scalar-iteration Eq/D calculus specified above. The change is in its finite certificate representation. A paired formula is unary, an equation is binary, and an internal closure is ternary:

$$\mathsf{D}_{x,y}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s,t], \qquad \mathsf{Q}_y B \longleftrightarrow [B,B,B]. \tag{Arity encoding}$$

The three copies in a closure are representation redundancy: only the third has mathematical content. Their benefit is that tripling a unary major constructs exactly its closure. Tripling any other nonempty formula array produces an array of arity at least six, which will denote truth. This makes the other major cases neutral without a separate test.

The previous generated-assumption and constructed-computation interfaces are inherited from the corpus [6, 10]. The new encoding makes a single arithmetic expression, $5-\text{arity}$, perform the necessary depth adjustments. Its soundness depends on an explicit interpretation of surplus syntax. It would not be sound to drop arity checks while continuing to treat every surplus array as an unconstrained new connective.

G.1 The complete sorted interpretation of arrays

Write $\top$ for the fixed equation $0 = 0$. A protected numerical slot has width two. The numerical root has depth three: the coordinates $0, 1, 2$ all denote zero, and under p protected slots the depth is $3 + 2p$. Coordinates $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot, for $0 \le j < p$. Formula depth is $2p$ under the same p slots. Negative coordinates admitted by the current global floor name rigid numerical parameters.

The decoder is sorted by the position in which a tree is used. Let ρ be its ordered environment of protected numerical slots. The notation (a, ρ) means prepend a slot of value a. A superscript $\sharp$ denotes a child decoded in the indicated environment.

Numerical arity	Meaning	Relevant child environment
0	0	No child
1: $[b]$	$Sb^\sharp$	$(0, 0, \rho)$
2	0	Annotations ignored
3: $[n, a, h]$	$I(n^\sharp, a^\sharp; v.h^\sharp)$	$(0, \rho)$ for n, a; (v, ρ) for h
≥ 4	0	All children ignored

Formula arity	Meaning	Relevant child environment
0	$\top$	No child
1: $[B]$	$\mathsf{D}_{x,y}B^\sharp$	(y, x, ρ)
2: $[s, t]$	$s^\sharp = t^\sharp$	ρ numerically
3: $[P, U, B]$	$\mathsf{Q}_y B^\sharp$	(y, ρ) for B
≥ 4	$\top$	All children ignored

The first two entries of a ternary formula are annotations. A canonical closure copies its body into both of them, as in (Arity encoding). Successfully admitted scalar formula padding also denotes $\top$. An internal closure has the ordinary meaning $B(0) \to \forall y B(y)$; it is not an equation and is not eligible for compound induction.

Canonical encoded object expressions use arrays and integer coordinates, respect these slots, and contain no high-arity node. The checker itself still generates high-arity truth entries in administrative contexts, even during a canonical proof. In particular, constructing a successor beneath positive bound coordinates requires passing through two dummy zero slots. Constructing an iterator requires one new slot: zero in its counter and seed, and its genuine state variable in the update. Rigid negative parameter names are unaffected by these shifts.

The additional raw shapes are certificate aliases with the displayed meanings. They are not independent predicate or function symbols added to the object theory. Our exact completeness statement concerns the canonical native language. We do not require every possible raw spelling of a semantic alias to be interchangeable under the same raw certificate.

G.2 One depth calculation replaces the lookup table

For every array the opener recursively adds $5 - \text{arity}$ to its depth. The meaningful cases are

arity	1	2	3
depth increment	4	3	2.

Unary D adds two numerical binders; unary successor adds two dummy zero slots. Binary equality changes formula depth $2p$ to numerical depth $3 + 2p$. Ternary Q adds one numerical binder; ternary iteration adds one state slot, or one dummy slot in its counter and initializer. The same calculation therefore covers all relevant constructors.

Binary numerical annotations denote zero, so their parity-changing traversal has no mathematical effect. Empty arrays have no children. At arity at least four, the entire node already denotes a constant. Even a zero or negative depth increment below such a node cannot expose one of its ignored children as a meaningful variable. Traversal may still reject an alias; it cannot change its stipulated interpretation.

This is a semantic assertion, not raw idempotence of the opener. For example, at floor -1, opening a six-element array of zeros with image -1 can yield six copies of -1, while a later no-image opening of that result fails at a negative child depth. Both formula arrays denote $\top$. The proof below only concerns successful calls at meaningful sorted positions. Canonical input object expressions contain no surplus arrays; the administrative exports that generate them are justified separately as truth entries.

G.3 Opening and scope

Write $i(a, t, d)$ for the executable opener at floor k. It preserves a scalar when $k \leq a < d$. At a numerical position, every other relevant scalar is the distinguished substitution hole; the supplied image is checked at numerical root depth three before insertion. At a formula position scalar padding is truth and carries no numerical hole. Missing images at used numerical holes reject.

Lemma G.1 (Semantic opening)**.** *For every successful call at a meaningful formula root of depth $2p$ or numerical root of depth $3 + 2p$, with $p \geq 0$, decoding its result is capture-free filling of the distinguished hole in the decoded input template. These depths protect p numerical slots; admitted rigid parameters are preserved. If an image is semantically unused, zero may be chosen as its mathematical representative. Recursive calls at negative depths inside ignored annotations need no independent sorted interpretation.*

Proof. Induct on the finite raw tree. At a relevant scalar, the interval comparison is exactly the protected-coordinate test. At a meaningful unary, binary, or ternary node, the preceding tables supply

precisely the required additional slots. At numerical binary annotations, high-arity nodes, and ignored closure fields, the relevant mathematical meaning is constant or absent, so the conclusion is immediate regardless of traversal inside the annotation.

An inserted image is checked at its own numerical root. Its free parameters are rigid negative names, and it has no unbound positive coordinate there. Insertion beneath protected slots therefore captures none of its parameters; its internal binders retain their relative indices. This proves the inductive step for substitution as well as for mere traversal. □

A successful no-image check consequently preserves the interpretation of its checked closed expression. Raw structural equality of checked outputs implies equality of their decoded meanings. The checker does not infer equality merely from a semantic comparison of two different programs or numerical terms.

The global floor decreases by one at each non-hypothesis continuation. If its old value is K, the integer $K-1$ is the designated fresh eigenparameter. With integer coordinates it is the only new name. If fractional coordinates are accepted, other names in $[K-1, K)$ may also occur in the finite certificate. These are additional fresh parameters. They must not be silently equated to the designated integer eigenparameter.

Previously checked relevant goals and retained hypotheses mention none of these new names: their earlier no-image checks would reject a relevant numerical occurrence below the old floor. Coordinates inside ignored annotations do not contribute free variables to the decoded formula. This is the freshness invariant used in every rule below.

G.4 A computation hypothesis only needs a zero instance

The shared continuation can open a generated arithmetic equation at a fresh parameter. That equation need not be a theorem uniformly in the new parameter. The following admissible rule is sufficient.

Lemma G.2 (Supply with a known zero instance)**.** *Suppose $\vec{a}$ is fresh for Γ, F, the calculus proves $H(\vec{0})$, and $\Gamma, H(\vec{a}) \vdash F$. Then $\Gamma \vdash F$.*

Proof. Substitute zero for every member of $\vec{a}$ in the second derivation. Freshness leaves Γ, F unchanged. Cut in the derivation of $H(\vec{0})$. Both substitution and cut are admissible in the native calculus. This is a finite proof transformation and does not use a consistency assumption. □

All names newly admitted when the floor changes belong to $\vec{a}$; for noninteger aliases this includes the finite collection in $[K-1, K)$ that actually occurs. On decoding, their simultaneous zero substitution agrees with opening the generated expression at zero using the old floor. Previously admitted parameters are preserved. New rigid names and unprotected placeholders both become zero. The opener may keep different raw zero spellings, but their decoded values agree.

The constructed-computation packet supplies a triple $s = [n, b, r]$ and a mode u. In successor mode its generated equation has raw form

$$\big[\,[[n], b, r],\ \ i(r, s, 3)\,\big], \tag{Step supply}$$

while in zero mode it has form

$$\big[\,[u, b, r],\ \ b\,\big]. \tag{Zero supply}$$

Here successor mode means that u is truthy; the zero mode has falsy u. These expressions describe actual arrays, before their final opening. In (Step supply) the counter is constructed as a successor,

rather than accepted on an unsupported assertion that it is one. This construction is inherited from the corresponding draft interface [10].

Lemma G.3 (Sound constructed computation)**.** *Every successful arithmetic supply has a decoded zero instance derivable from the two scalar-iteration computation schemas.*

Proof. First apply the simultaneous zero substitution described above. In zero mode every successful falsy numerical counter denotes zero, so (Zero supply) is an $I0$ instance. In the exact-list profile the relevant accepted aliases are an empty array, numeric zero, false, and numerically equal zero values. Null, strings, and dictionaries cannot enter a meaningful numerical position through an unchecked coercion.

In successor mode the left counter is an actual successor. If the update uses its state in a semantically relevant position, opening it invokes the no-image check of the recursive source s at numerical root three. This checks the entire source, including its update. Only coordinates three and four are aliases for its single state slot. A genuinely escaping coordinate in that update is revisited during the recursive-image check and rejects. Its counter and initializer are likewise closed with their prescribed dummy zero slot.

Inserting the extra dummy slots of the displayed successor does not alter such a checked predecessor: extra unbound slots have value zero, and actual free parameters are rigid. Lemma G.1 therefore identifies the right side of (Step supply) with the update applied to exactly the recursive predecessor value occurring on its left. This is the IS schema.

If the update has no semantically relevant state occurrence, its decoded value is constant in that variable. Its zero-normalized value is the same in the source update and on the right, independently of the unused recursive image; this is again an IS instance. Traversal inside an ignored annotation may still inspect the image and reject. That removes a certificate and does not invalidate the argument for a successful one.

Finally, successful three-field source extraction cannot hide another JSON operator. A scalar is not iterable. A three-character string, or a three-key JSON dictionary, necessarily puts a nonempty string in a relevant source field, which the inspected opener rejects. The successful source is therefore the stated array triple. □

The recursive-image check in the state-dependent case is essential. Merely opening a final displayed equation at zero would not establish that its two state interpretations agree. Combining Lemma G.3 with Lemma G.2 justifies both actual image choices of the common continuation, including the branch that retains a fresh-parameter hypothesis.

G.5 Audit of the logical continuations

Hypothesis closure. The initial opener check and membership comparison select the same decoded context formula. Additional aliases can make a spelling less convenient, but cannot identify an equation with a false denotation solely by semantic evaluation.

Equality transport. The major must match $[s, u]$, and opening the proposed formula template at u must match the target. The continuation checks its opening at s. Lemma G.1 gives ordinary transport along the decoded equality. The argument applies to compound formulas, internal clauses, and truth aliases as well as equations.

Closure specialization. The closure pattern is the triplicate $[t, t, t]$. The guard forces the old instance to have numerical image zero; the new instance may have an arbitrary numerical image. This is exactly $B(0) \to \forall y B(y)$. The zero guard is indispensable. Without it, a closure with false zero instance and true successor instances would turn a proved successor instance into falsity.

Paired elimination. The export is always three concatenated copies of the major. A unary major $[B]$ therefore exports $[B, B, B]$. All three copies are opened identically; the third copy has precisely the witness instance required by paired elimination. The newly allocated witness is fresh for the retained context and conclusion.

An equation major instead exports arity six; a closure major exports arity nine; a larger array exports still larger arity. These are all truth aliases. Thus failed elimination endpoint claims have only the effect of adjoining a theorem. A numeric scalar major, if accepted as formula padding, exports scalar padding after multiplication and also denotes truth. Unsupported types reject.

A ternary formula with unequal annotations still denotes the closure of its third child. It fails the triplicate specialization pattern unless the three fields match. This loses a possible raw spelling, not a mathematical soundness condition. Canonical compilation uses triplicate closures throughout.

Paired introduction. The call `i(*a,t,2)` uses the function's argument count to enforce the relevant outer arity. For a unary goal $a = [B]$, it is exactly `i(B,t,2)`: the universal slot is protected while the existential witness is opened. For arity at least two, there are too many arguments and the call rejects. An empty goal can instead make the legal two-argument call `i(t,2)`. Its original target was already $\top$, so accepting it cannot add a false theorem.

For the unary goal, the subsequent successor opening gives the intended premise $B(t, Sz)$. The local hypothesis is $B(t, 0)$ in case mode and $B(t, z)$ in progress mode. The freshness invariant supplies the usual eigenvariable conditions.

Induction remains restricted. Among meaningful nonconstant matrix constructors, only binary equality has even arity. Unary paired formulas and ternary closures have odd arity and receive case introduction. Empty or surplus arrays that reach the progress branch denote the fixed equation $0 = 0$, whose induction is valid. Scalar padding that cannot provide a length rejects. The broader certificate alphabet therefore adds no induction on a quantified matrix.

Additional fresh parameters. An accepted subderivation can contain newly admitted fractional names, but the older context and target do not depend on them. Replace these auxiliary names by zero when closing the rule, retaining its designated integer eigenvariable where that rule needs one. The common opening of an already checked retained goal leaves its decoded meaning unchanged. This handles the actual global-floor convention without pretending that every admissible negative number is an integer.

G.6 Canonical completeness, including zero witnesses

Rename a finite native proof's eigenvariables to fresh negative integers. Encode its terms with the slot conventions of Section G.1; encode paired formulas as unary and closures as triplicate. Hypothesis leaves consume no instruction. Transport, closure specialization, and paired elimination have their displayed packets. Arithmetic computation leaves use (Zero supply) or (Step supply) and then close the target by hypothesis lookup.

One representational issue requires an actual proof transformation. The packet's truthy witness field selects introduction. A literal zero witness therefore cannot simply be inserted into that field. Define

$$J(t) = I(0, t; v.0).$$

The computation axiom proves $J(t) = t$, and $J(t)$ has a nonempty array representation even when its value is zero. To introduce with a falsy witness spelling t, introduce instead with $J(t)$: transport its retained assumption to the assumption using t, use the original premise derivation, and transport the resulting successor instance back to the instance using $J(t)$. Derived symmetry supplies the reverse transport. This is a finite transformation of the premise, and the paired conclusion itself is unchanged. If the existential coordinate is unused, any closed truthy witness works without a conversion.

The scalar coordinate 1 is also a semantic zero at numerical root three, but that observation alone is not a completeness proof. Closure specialization and case introduction insert literal raw zero, and distinct zero aliases need not be interchangeable under a fixed raw proof spelling. The computation-backed box gives the required derivation instead of assuming representational interchangeability.

The mode zero selects progress at an equation. A different falsy mode, such as the empty array, selects case. These are certificate control tags, not numerical assertions about the witness. All steps in the compilation are finite and effective.

Theorem G.4 (Conservativity and completeness of the arity checker)**.** *Every accepted certificate has its decoded conclusion provable in the ordinary realization of the native calculus. For every canonical native end sequent, the internal checking judgment and the native calculus have the same consequence relation, in the idealized unbounded integer-and-array implementation. In particular, the public empty-context driver recognizes exactly the canonical closed theorems. The translation in each direction is effective on finite derivations.*

Proof. For soundness, induct on the finite accepting call tree. Hypothesis closure uses Lemma G.1; the logical cases are the audited continuations above. Arithmetic supply uses Lemmas G.3 and G.2. Every discarded annotation is accounted for by its zero or truth interpretation, and every fresh parameter by the scope invariant. For completeness, compile a finite native derivation by the preceding construction, including the boxed-witness transformation. Its canonical input expressions require no surplus-arity aliases or fractional names; generated administrative surplus arrays have exactly the truth meaning already proved. This proves both implications without using a finite test suite as a substitute for induction on derivations. □

The unchanged native calculus is bi-interpretable with full first-order iPRA by the inherited arithmetic and logical bridge [6, 12]. In particular,

$$(\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

The whole numerical domain and actual equality are retained. Its internally proved numerical round trips and its formula-composite derivations therefore transfer through Theorem G.4. We have changed the checked representation, not replaced bi-interpretability by standard-model agreement or supplied a new proof of the scalar predecessor theorem.

G.7 JSON and finite-runtime qualifications

The primary profile uses an ordinary list-type test. The smaller byte profile instead tests multiplication by zero against an empty list. Among standard JSON runtime values the latter recognizes exactly lists: numeric values and booleans produce numeric zero, strings produce an empty string, and null

and dictionaries raise rather than becoming arrays. This argument assumes ordinary decoded JSON values, not arbitrary Python objects with overloaded methods.

Finite nonnegative real coordinates have the same interval interpretation: $[0, 3)$ is zero, and $[3 + 2j, 5 + 2j)$ names slot j. Finite negative coordinates in the admitted range are additional rigid names. Equal Python numerical values have the same decoded role, including Boolean aliases and signed zero. Nonfinite values admitted by Python's permissive JSON reader cannot survive a relevant closed numerical check. They may be filled as template holes or ignored inside a constant annotation; neither makes them unchecked parameters.

The alternative opener based on generic iteration has a separate runtime qualification. Empty strings and empty dictionaries traverse as empty arrays; a dictionary whose sole key is empty traverses as a unary array with an empty child. Dictionary values are not visited. Nonempty strings can rely on CPython's recursion limit to reject, as also occurs in a current corpus comparison profile [11]. The idealized structural-termination claim for the exact-list profile must not be transferred to that generic profile unchanged.

All executable profiles have ordinary finite memory, stack, and input limits. These can reject a sufficiently large otherwise valid proof. The formal consequence relation is the unbounded finite-certificate calculus; the delivered program checks certificates within its runtime resources. The measurements charge the entire delivered checker, while the semantic proof and canonical producer are supplied separately.

References

[1] Vladimir Reshetnikov and ProveIt contributors, *ConciseIPRA*, README and research corpus at the pinned commit. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt, *source_dispatch*, especially `proofs/zero_source_admissibility.tex`. The same source is included in `reference/foundations/zero_source_admissibility.tex`.

[3] ProveIt, *guarded_core*, especially `sections/guarded_core.tex`; reproduced in Appendix B.

[4] ProveIt, *certified_zero/reference*, the inherited choice basis, ordinary bridge, scalar predecessor, equality reflection, primitive recursion, and arity checker.

[5] ProveIt, *ConciseIPRA Lean development*, including `BiInterpretation.lean`, `NativeBiInterpretation.lean`, and `NativeZeroSource.lean`, at the pinned commit. The sources were read; the upstream project was not rebuilt for this package.

[6] ProveIt research series, the generated-assumption and Choice-DI presentation; preserved in `reference/foundations/inherited_choice_basis.tex`.

[7] ProveIt research series, scalar predecessor development attributed in the inherited source to the unparenthesized one-hole report; preserved in `reference/foundations/scalar_predecessor.tex`.

[8] ProveIt research series, equality-reflection development attributed in the inherited source to the forward-left report; preserved in `reference/foundations/equality_reflection.tex`.

[9] ProveIt research series, progress-closure arithmetic and primitive-recursion compiler; preserved in `reference/foundations/primitive_recursion.tex`.

[10] ProveIt, *constructed_computation*, `Research/constructed_computation/` in the pinned tree.

[11] ProveIt, *transient_seed*, `Research/transient_seed/` in the pinned tree.

[12] ProveIt research corpus, the filed packages and incoming archives in the pinned ConciseIPRA tree. This compilation preserves the original mathematical text and its attribution; the present measurements refer to the exact standalone modules in the main report.